Functional Requirements

1. User Management

FR-1: Users shall be able to register an account.

FR-2: Users shall be able to log in using authenticated credentials.

FR-3: The system shall assign roles to users (Example: Customer, Service Provider, Admin).

FR-4: Users shall be able to view and update their profiles.

FR-5: The system shall store user profile information.

FR-6: The system shall allow users to deactivate their accounts.

2. Business Management

FR-7: Business owners shall be able to register a business on the platform.

FR-8: Business owners shall be able to update business information.

FR-9: Business owners shall be able to define services offered by their business.

FR-10: Business owners shall be able to manage business availability schedules.

FR-11: The system shall associate businesses with their owners.

FR-12: Customers shall be able to view available businesses and services.

3. Booking Management

FR-13: Customers shall be able to create bookings for available services.

FR-14: The system shall verify availability before creating a booking.

FR-15: The system shall prevent conflicting bookings.

FR-16: Newly created bookings shall have a default status of **Pending**.

FR-17: Business owners shall be able to approve bookings.

FR-18: Business owners shall be able to reject bookings.

FR-19: Customers shall be able to view their booking history.

FR-20: Business owners shall be able to view bookings associated with their businesses.

FR-21: The system shall maintain booking status history.

4. Notification Management

FR-22: The system shall generate notifications when bookings are created.

FR-23: The system shall notify business owners of new booking requests.

FR-24: The system shall notify customers when bookings are approved.

FR-25: The system shall notify customers when bookings are rejected.

FR-26: The system shall support email notifications.

FR-27: The system shall support SMS notifications.

FR-28: The system shall maintain notification delivery records.

5. Security and Authentication

FR-29: The system shall authenticate users using OAuth2.

FR-30: The system shall issue JWT access tokens after successful authentication.

FR-31: The system shall authorize access based on user roles.

FR-32: Unauthorized users shall be denied access to protected resources.

FR-33: All requests to backend services shall pass through the API Gateway.

6. Service Discovery and Configuration

FR-34: Microservices shall register with the service registry.

FR-35: Services shall discover other services dynamically.

FR-36: Services shall retrieve configuration from the centralized Config Service.

7. Future Payment Support (Optional)

FR-37: The system shall support integration with payment providers.

FR-38: The system shall record payment transactions associated with bookings.

FR-39: The system shall support payment confirmation notifications.

Non-Functional Requirements

1. Performance

NFR-1: The system shall respond to standard API requests within 2 seconds under normal operating conditions.

NFR-2: Booking availability checks shall complete within 1 second under normal load.

NFR-3: Notification processing shall occur asynchronously and shall not block booking creation.

2. Scalability

NFR-4: The system shall support horizontal scaling of individual microservices.

NFR-5: The architecture shall support deployment in containerized environments.

NFR-6: The architecture shall support future migration to Kubernetes.

3. Availability

NFR-7: The platform shall target 99.5% service availability.

NFR-8: Failure of one microservice shall not cause complete platform failure.

NFR-9: The system shall continue processing requests when non-critical services are unavailable.

4. Reliability

NFR-10: Booking data shall be stored using ACID-compliant transactions.

NFR-11: The system shall ensure booking consistency and prevent double booking.

NFR-12: Messages published to RabbitMQ shall be delivered reliably.

NFR-13: Failed notifications shall support retry mechanisms.

5. Security

NFR-14: All communication shall occur over HTTPS.

NFR-15: User passwords shall never be stored in plain text.

NFR-16: JWT tokens shall be validated before granting access.

NFR-17: Role-based access control shall be enforced.

NFR-18: API Gateway shall provide centralized security enforcement.

NFR-19: Rate limiting shall protect APIs against abuse.

6. Maintainability

NFR-20: Services shall be independently deployable.

NFR-21: Services shall follow clean architecture and separation of concerns.

NFR-22: APIs shall be documented using OpenAPI/Swagger.

NFR-23: The codebase shall support automated testing.

7. Observability

NFR-24: The system shall generate centralized application logs.

NFR-25: The system shall expose health-check endpoints.

NFR-26: The architecture shall support distributed tracing.

NFR-27: The architecture shall support metrics collection and monitoring.

8. Usability

NFR-28: The user interface shall be responsive across desktop and mobile devices.

NFR-29: Users shall be able to complete a booking with minimal steps.

NFR-30: Error messages shall be meaningful and understandable.

9. Interoperability

NFR-31: Services shall communicate using REST APIs over HTTP.

NFR-32: Service contracts shall exchange data using JSON.

NFR-33: The platform shall support integration with third-party email and SMS providers.

10. Data Management

NFR-34: Each microservice shall own and manage its own database.

NFR-35: User data shall be stored in MongoDB.

NFR-36: Business, Booking, and Notification data shall be stored in PostgreSQL.

NFR-37: Data persistence shall survive container restarts through persistent storage volumes.